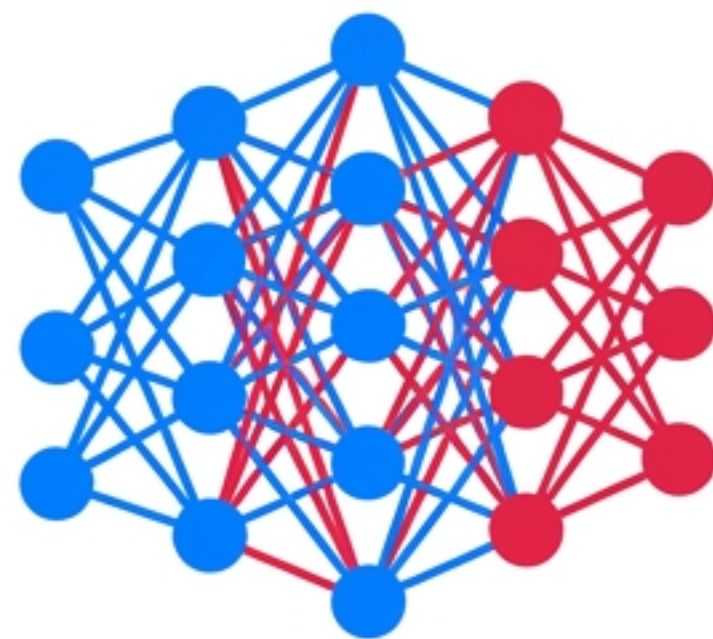




Giới thiệu bậc đại học môn Máy học

Giới thiệu bậc đại học môn Máy học

Chương 5 - Phần 2: Tối ưu hóa Mô hình & Kỹ thuật Regularization

KHÔNG GIAN THIẾT KẾ MÔ HÌNH: TỪ DUNG LƯỢNG ĐẾN KỸ THUẬT TÍNH NĂNG

Xử lý khi Validation Loss không chịu giảm



- **Triệu chứng:** Loss không giảm, Accuracy kẹt ở mức chọn bừa (Random Guess).
- **Nguyên nhân:** Learning Rate quá khổng lồ (ví dụ: 1.0), khiến bước Gradient bay nhảy hỗn loạn.

- **Giải pháp:** Hạ Learning Rate (ví dụ: $1e-2$) để giảm trơn tru. Tăng Batch Size để cung cấp định hướng đạo hàm chính xác, ít nhiễu.

```
model.compile(  
    optimizer=keras.optimizers.RMSprop(  
        learning_rate=1e-2),  
    loss="sparse_categorical_crossentropy",  
    metrics=["accuracy"],  
)  
model.fit(train_images, train_labels,  
    epochs=10, batch_size=128)
```

Dung lượng Mô hình (Model Capacity) là gì?

Quá Nhỏ (Underfitting)



Không đủ khả năng 'ghi nhớ' quy luật phi tuyến tính phức tạp.

Lý Tưởng (Mục Tiêu)



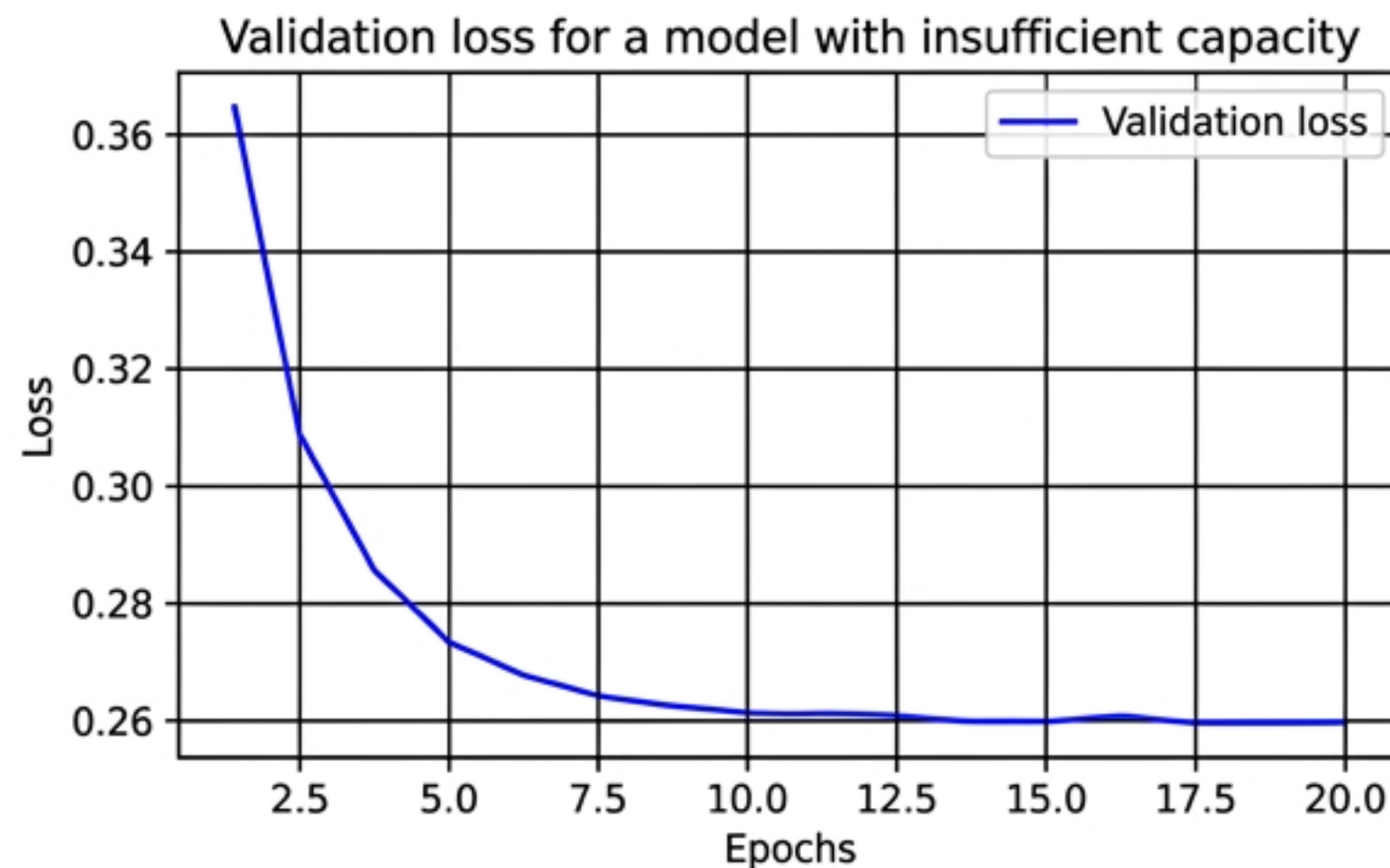
Giảm loss tối đa nhưng trì hoãn việc overfit muộn nhất. Vừa khít.

Quá Lớn (Overfitting)



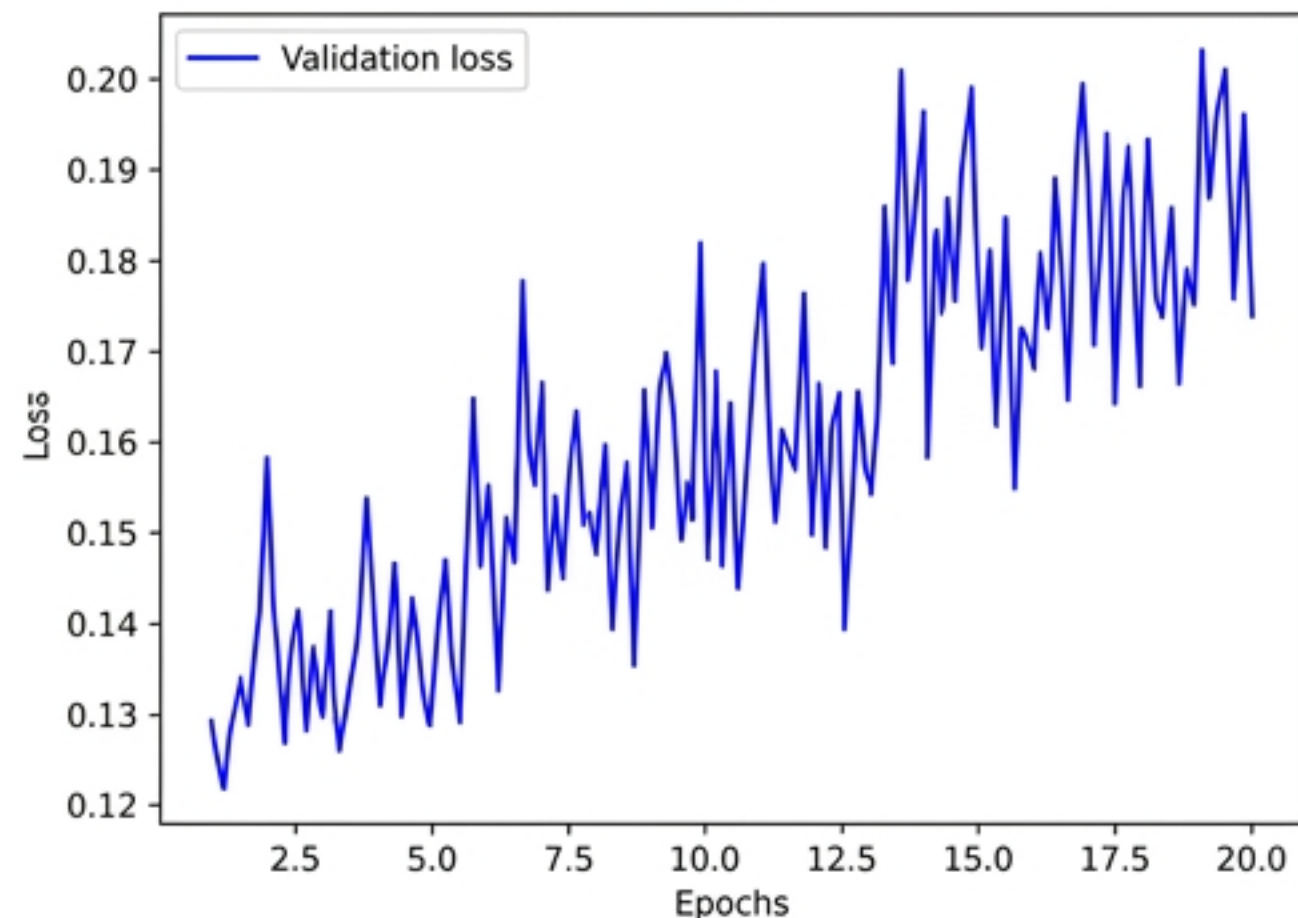
Dư thừa, học vẹt cực nhanh (mapping trực tiếp input ra output).

Khi Dung lượng Mô hình Không Đủ



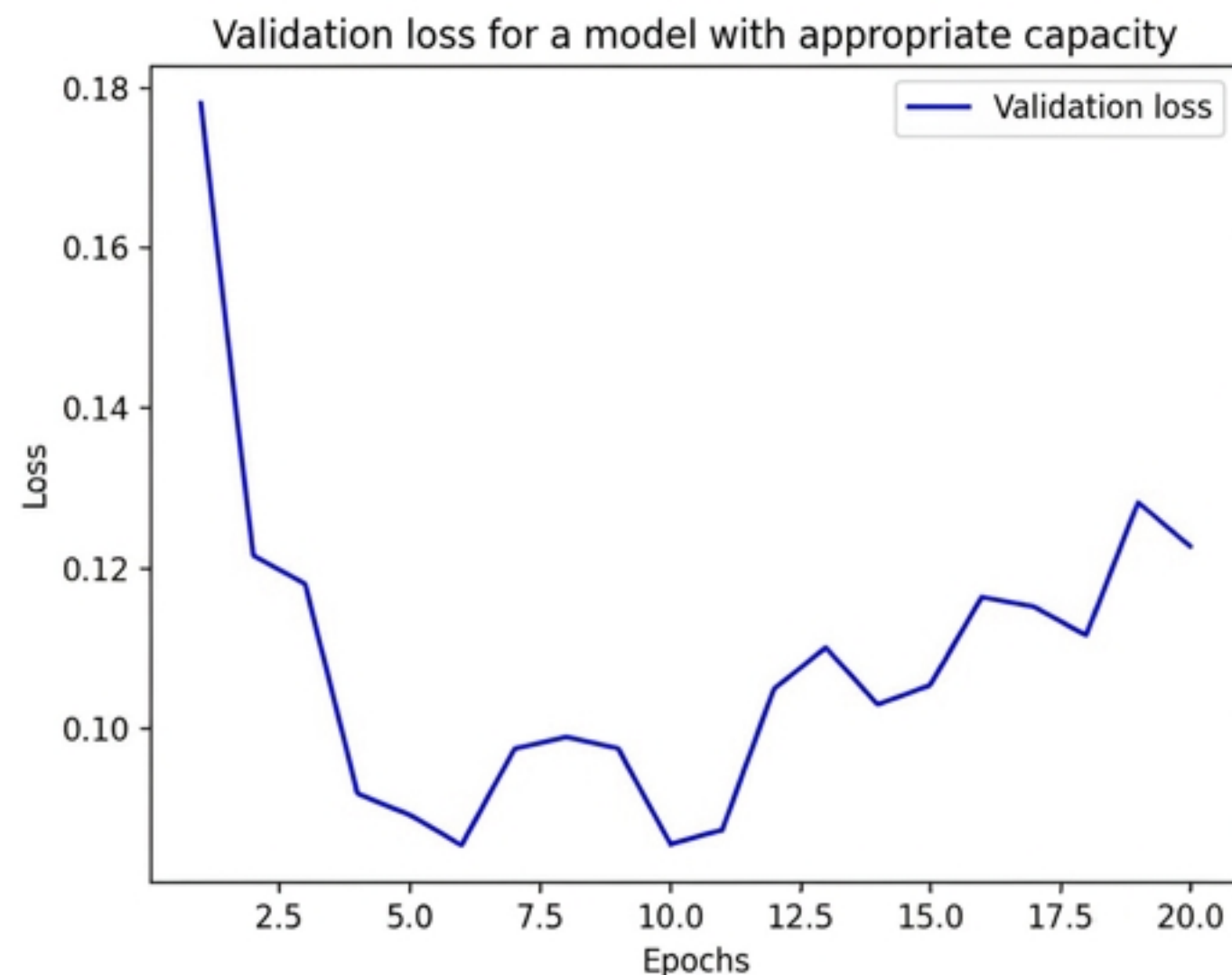
- Thử nghiệm: Mạng hồi quy Logistic siêu nông trên MNIST.
- Triệu chứng: Validation Loss chững lại ở mức 0.26 và giữ nguyên (Stalled).
- Kết luận: Mô hình không đủ sức mạnh để khái quát không gian giả thuyết phức tạp.

Khi Dung lượng Mô hình Quá Lớn



- **Thử nghiệm:** Mạng khổng lồ (3 lớp x 2048 nơ-ron).
- **Triệu chứng:** Ghi nhớ thần tốc. Nhớ toàn bộ tập Training trong vài giây.
- **Kết quả:** Validation Loss chạm đáy ngay tại Epoch 1, sau đó vọt lên mạnh mẽ với biên độ siêu nhiều (noise).

Dung lượng Mô hình Lý Tưởng



- Thiết kế: 2 lớp ẩn (mỗi lớp 128 units).
- Kết quả: Validation Loss giảm từ từ và duy trì ở mức tối thiểu, chỉ tăng nhẹ ở epoch thứ 8.
- Đánh giá: Kiến trúc cân bằng tuyệt vời giữa tối ưu hóa và khái quát hóa.

So sánh Dung lượng: Bài toán IMDB



Mạng Gốc

```
layers.Dense(16, activation="relu")  
layers.Dense(16, activation="relu")
```

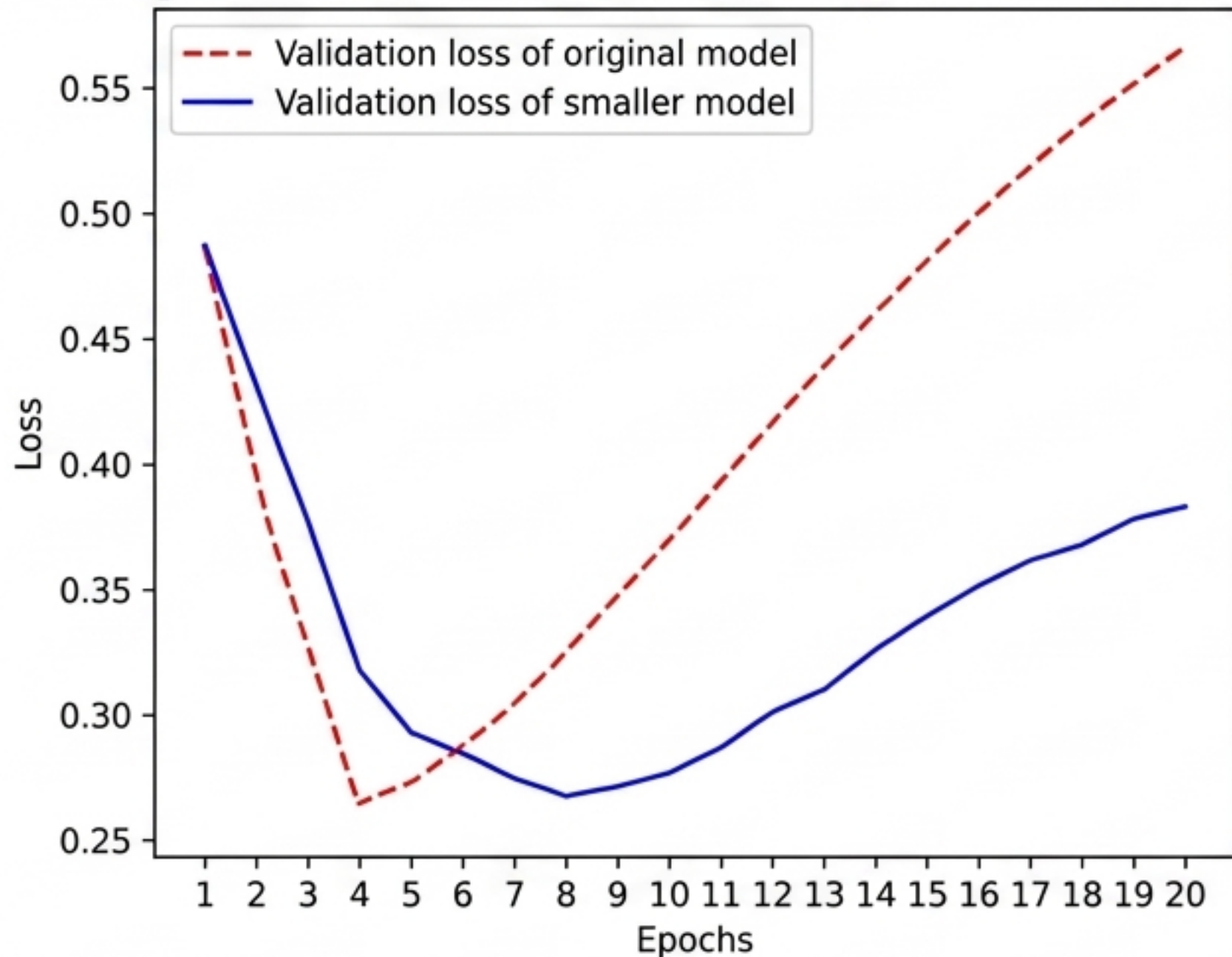
Mạng Nhỏ

```
layers.Dense(4, activation="relu")  
layers.Dense(4, activation="relu")
```

Thử nghiệm đẩy kiến trúc từ 16 units xuống 4 units (để giảm thiểu Overfitting), và vọt lên 512 units.

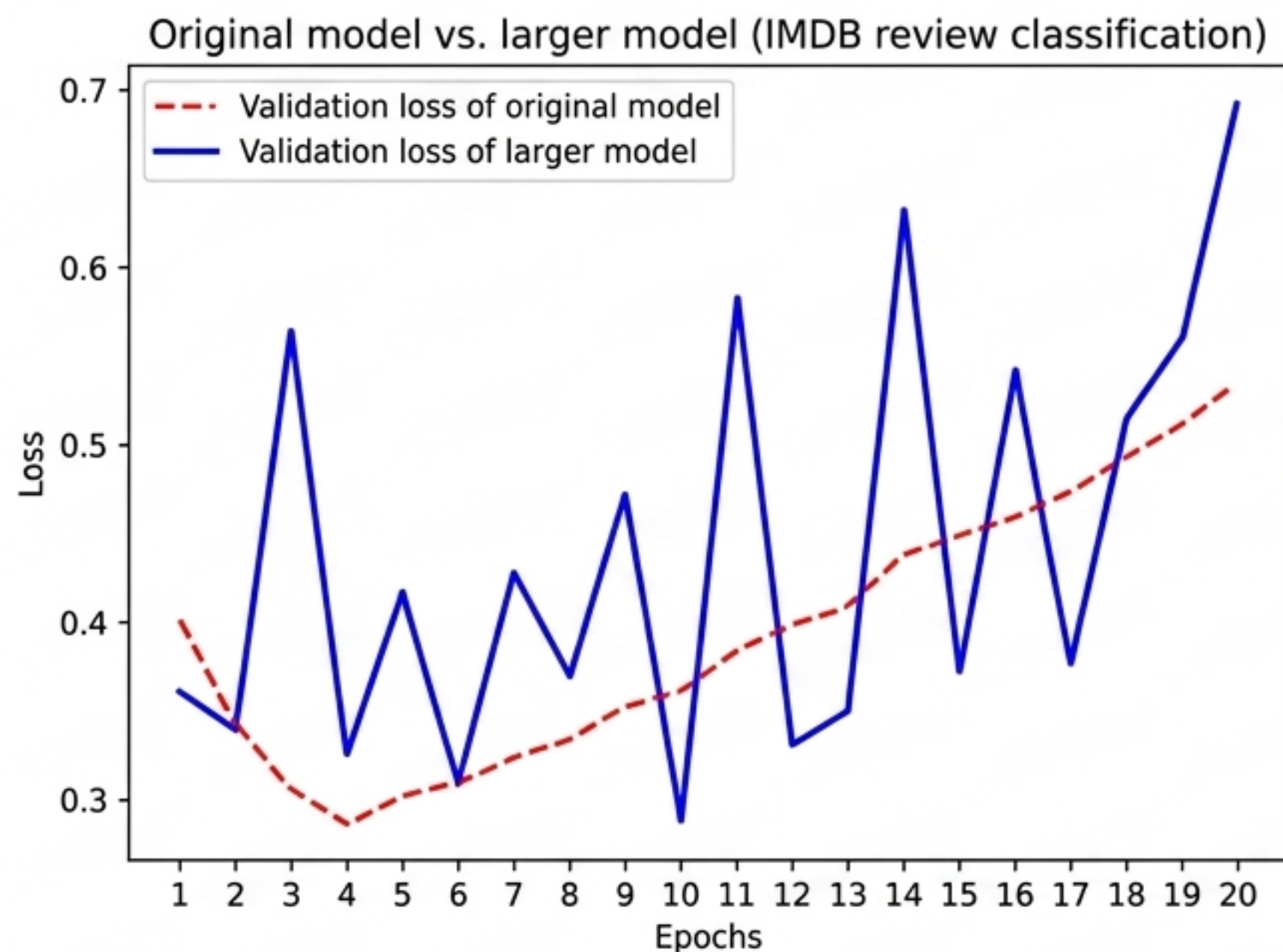
Trực quan: Mạng Gốc (16 units) vs Mạng Nhỏ (4 units)

Original model vs. smaller model (IMDB review classification)



- **Đường Dots (Chấm tròn):** Mạng gốc 16 units.
- ✕ **Đường Solid/Crosses (Đứt khúc):** Mạng nhỏ 4 units.
- ➡ **Độ trễ Overfit:** Mạng nhỏ mất thêm 6 epoch mới bắt đầu tăng Validation Loss.
- ✓ **Kết luận:** Giảm dung lượng mạng chính là kỹ thuật Regularization gốc rễ.

Trực quan: Mạng Gốc (16 units) vs Mạng Lớn (512 units)



- **Đường Solid/Crosses:** Mạng khổng lồ 512 units.
- **Hiện tượng:** Rơi vào tình trạng "Ghi nhớ thần tốc".
- **Hậu quả:** Overfit dữ dội ngay từ epoch đầu tiên. Biên độ Loss bị nhiễu cực lớn.

Phạt Trọng Số (Weight Regularization) L1 & L2

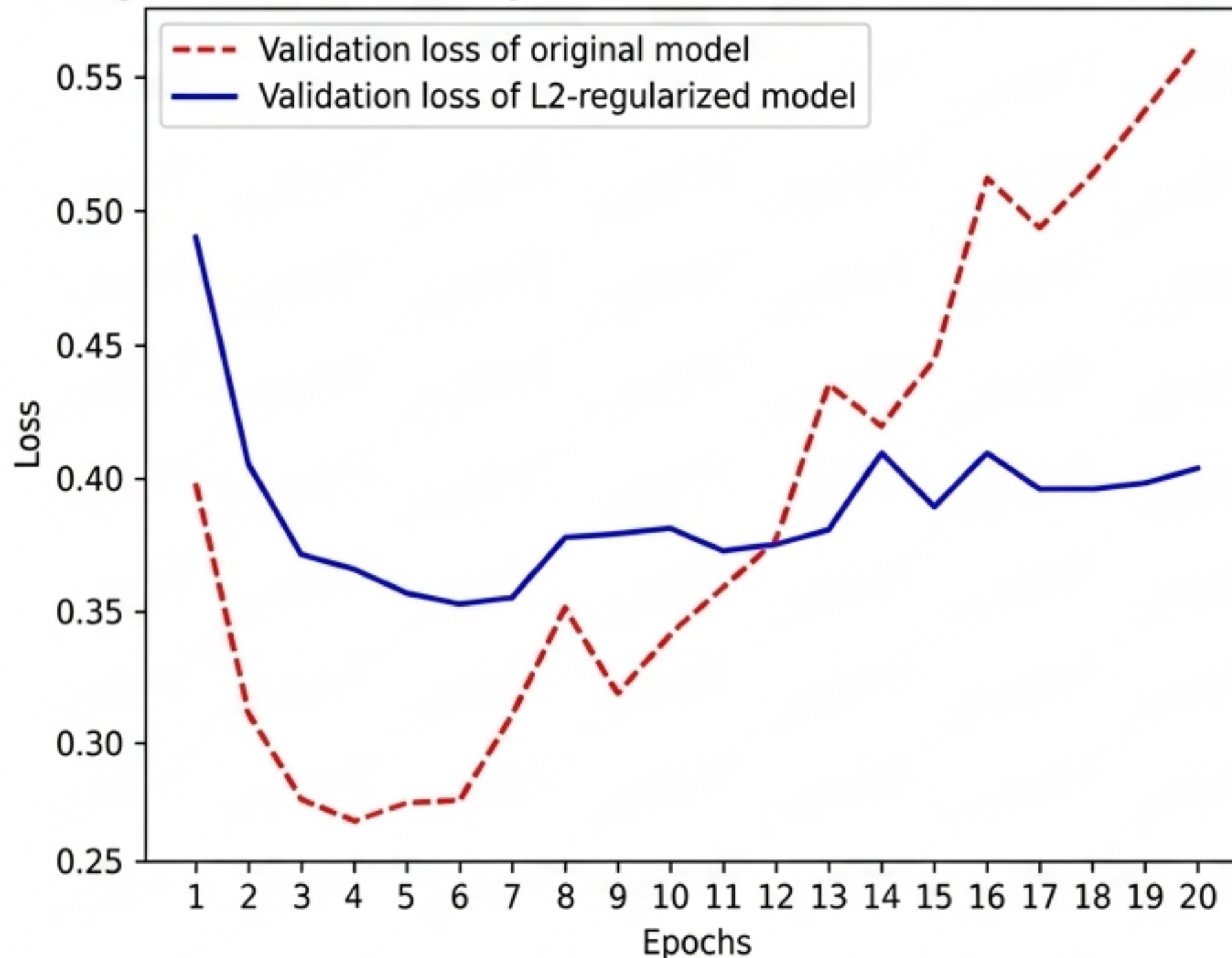


L1 Regularization	Thêm mức phạt tỷ lệ với trị tuyệt đối $ W $. Ép trọng số về 0 (Tạo sự thưa thớt - Sparsity).
L2 Regularization / Weight Decay	Thêm mức phạt tỷ lệ với bình phương W^2 . Ép trọng số tiệm cận 0.

```
layers.Dense(16, kernel_regularizer=l2(0.002))
```

Hiệu quả của Phạt trọng số L2

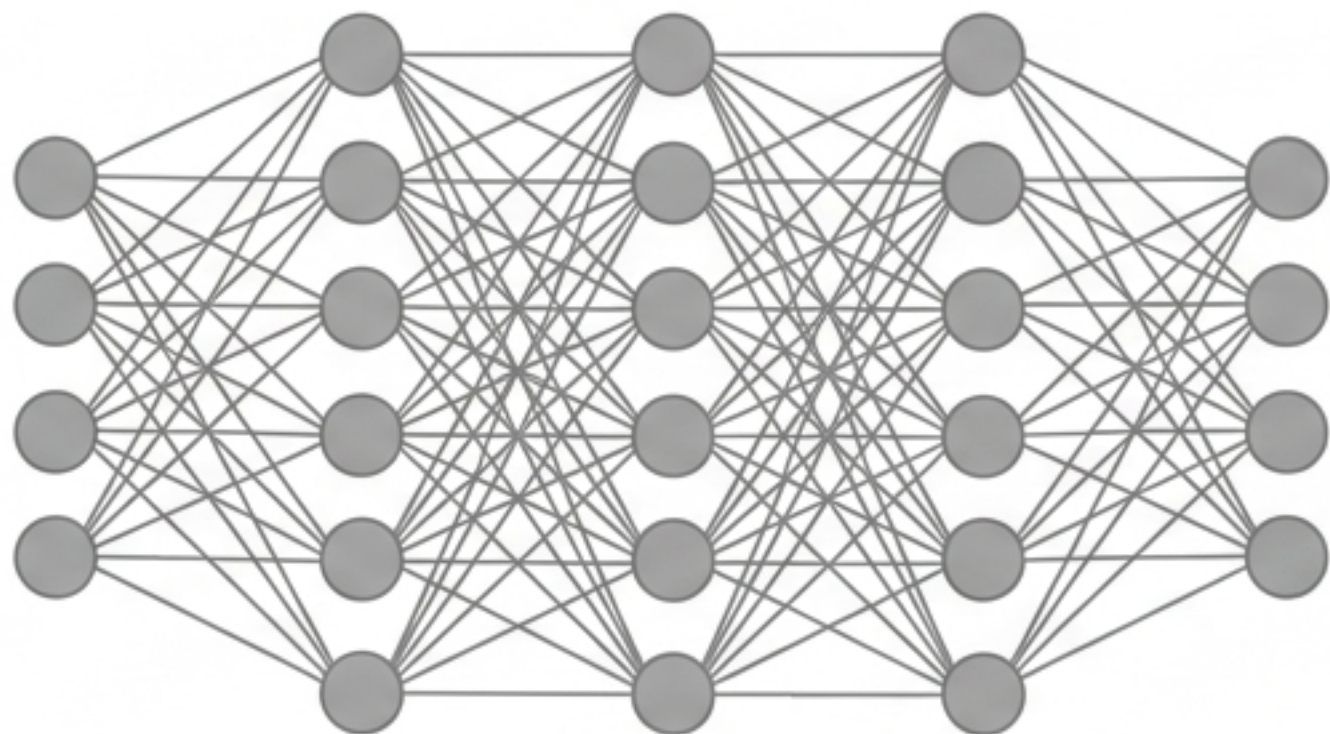
Original model vs. L2-regularized model (IMDB review classification)



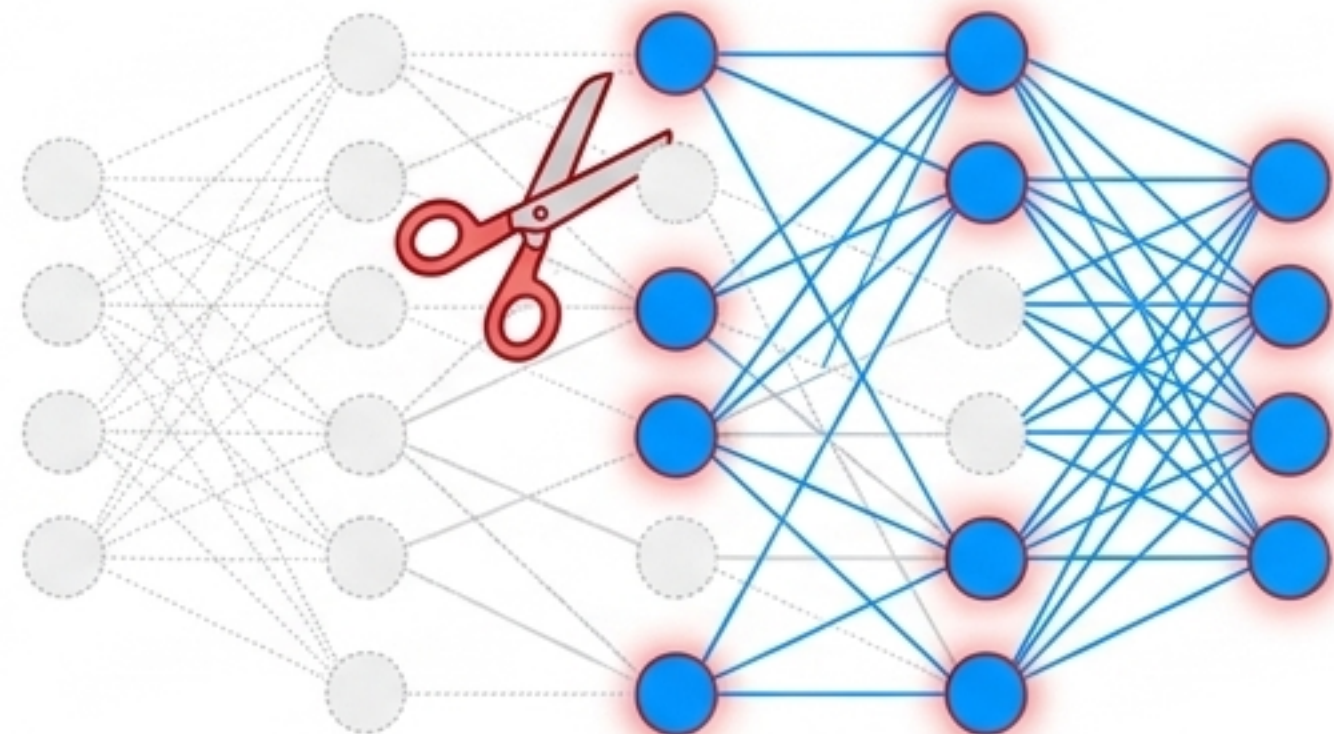
- **Cơ chế hoạt động:** Khoản phạt $0.002 \times W^2$ chỉ cộng vào hàm **Loss** lúc Huấn luyện. Nó tắt hoàn toàn lúc Kiểm tra.
- **Hiệu quả (Đường Crosses):** Duy trì Validation Loss ở mức an toàn, kháng **Overfit** lâu hơn đáng kể so với mạng gốc.
- **Ứng dụng:** Hoạt động tốt nhất trên các mô hình có **kích cỡ bé và vừa**.

Kỹ thuật Dropout (Ngắt Nơ-ron Ngẫu nhiên)

Mạng Gốc



Với Dropout



- **Phát minh:** Kỹ thuật cách mạng do Geoffrey Hinton giới thiệu.
- **Cơ chế:** Tại mỗi bước tối ưu, đột ngột vô hiệu hóa ngẫu nhiên (ép về 0) một tỷ lệ nơ-ron (thường $0.2 \sim 0.5$).
- **Mục đích:** Ngăn chặn "đồng thích nghi" (Co-adaptation). Nơ-ron buộc phải tự hình thành đặc trưng độc lập, không ỷ lại nơ-ron khác.

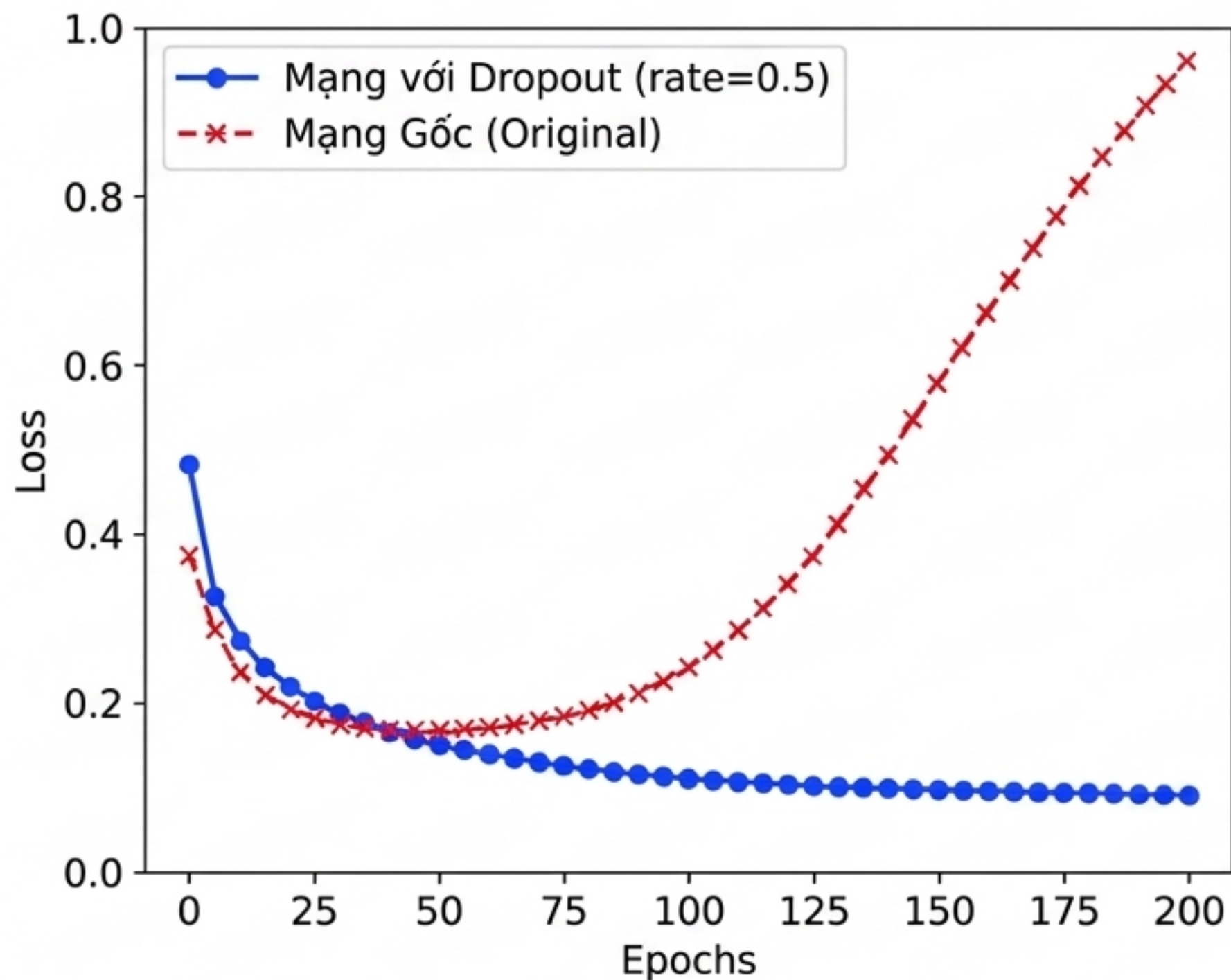
Cơ chế cân bằng Toán học của Dropout



- **Vấn đề:** Lúc Training ngắt 50% (rate = 0.5). Lúc Testing chạy full 100%. Tổng lực (Activation) tăng gấp đôi sẽ làm vỡ hàm Loss!
- **Giải pháp (Keras):** Tại bước Training, các nơ-ron sống sót tự động được Scale Up (nhân cho $1 / (1 - 0.5) = 2$).

```
layers.Dropout(0.5) # Nhét ngay sau lớp Dense
```

Hiệu quả thực tế của Dropout



- Mạng sử dụng **Dropout rate = 0.5** (Đường Solid/Crosses).
- **Kết quả:** Kháng cự lại sự **Overfit** vô cùng kiên cường.
- **Tầm quan trọng:** Đây là kỹ thuật **Regularization** hiệu quả bậc nhất, được áp dụng gần như mặc định cho các mô hình **Deep Learning** cực sâu.

Lựa chọn thay thế: Kỹ thuật Tính năng (Feature Engineering)



- **Định nghĩa:** Sử dụng kiến thức chuyên gia (**Domain Knowledge**) biến đổi dữ liệu đầu vào thân thiện hơn với thuật toán.
- **Lợi ích:** Dù DL có thể tự trích xuất tính năng ẩn, nhưng với dữ liệu nhỏ/đặc thù, thiết kế tốt giúp mô hình: Tốn ít tài nguyên, chạy nhanh hơn, chính xác hơn.
- **Bài toán kinh điển:** Nhận diện thời gian trên đồng hồ kim. Trích xuất góc kim đồng hồ trực tiếp thay vì bắt mạng nơ-ron tự học từ pixel thô.